TaskForge — Day 3 Notes

Day 3: PostgreSQL Persistence with FastAPI & SQLAlchemy

Goal: Replace temporary in-memory job storage with PostgreSQL so jobs remain available after the API server restarts.

Day 3 outcome: Connected FastAPI to PostgreSQL, created a SQLAlchemy `Job` model and database table, moved the job endpoints to database sessions, and verified that a job survives a server restart.

---

1. Why persistent storage?

Earlier, TaskForge stored jobs in a Python dictionary: `jobs = {}`. That data existed only while the Python process was running. When the server stopped, the dictionary disappeared.

PostgreSQL stores records durably, so they remain available after the application restarts.

Architecture

```
FastAPI endpoints
  Receive HTTP requests
        ↓
SQLAlchemy ORM + Session
  Translate Python operations into database queries
        ↓
PostgreSQL — taskforge database
  Stores records in the jobs table
```

---

2. Install and verify dependencies

SQLAlchemy is the ORM; Psycopg 3 is the PostgreSQL driver. Both were installed inside the project virtual environment.

```
python -m pip install sqlalchemy "psycopg[binary]"
```

PostgreSQL was installed with Homebrew and checked with:

```
brew --version
psql --version
pg_isready
```

The `taskforge` database was created and verified with:

```
psql postgres
CREATE DATABASE taskforge;
\q
psql -d taskforge -c "SELECT current_database();"
```

---

3. Configure the engine and session

File: `app/database.py`

```
from collections.abc import Generator
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session

DATABASE_URL = "postgresql+psycopg://granth@/taskforge?host=/tmp"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
)


class Base(DeclarativeBase):
    pass


def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Key concepts

- `DATABASE_URL` tells SQLAlchemy which database and driver to use. This URL matches the local Mac configuration used in this session.
- `engine` manages database connectivity.
- `SessionLocal` creates database sessions.
- `Base` is the parent class for ORM models.
- `get_db()` provides a session to an endpoint and closes it in `finally`.

---

4. Define the Job model

File: `app/models.py`

```
import uuid
from typing import Any
from sqlalchemy import String, JSON
from sqlalchemy.orm import Mapped, mapped_column
from .database import Base


class Job(Base):
    __tablename__ = "jobs"

    id: Mapped[str] = mapped_column(
        String(36), primary_key=True,
        default=lambda: str(uuid.uuid4()),
    )
    type: Mapped[str] = mapped_column(
        String(255), nullable=False
    )
    payload: Mapped[dict[str, Any]] = mapped_column(
        JSON, nullable=False
    )
    status: Mapped[str] = mapped_column(
        String(20), nullable=False, default="queued"
    )
```

A SQLAlchemy model maps a Python class to a database table. Each `Job` object represents one row. The ID is a UUID string; `payload` is stored as JSON; `status` defaults to `queued`.

---

5. Create the PostgreSQL table

Defining a model does not create a table by itself. SQLAlchemy metadata was used to create the table:

```
python -c "
from app.database import engine, Base
from app.models import Job
Base.metadata.create_all(bind=engine)
print('Jobs table created')
"
```

The table was verified with:

```
psql -d taskforge -c "\dt"
```

The output showed `public.jobs`, owned by the local PostgreSQL role `granth`.

---

6. Connect FastAPI endpoints to PostgreSQL

The in-memory dictionary was removed. Each endpoint receives a SQLAlchemy session through FastAPI dependency injection:

```
db: Session = Depends(get_db)
```

FastAPI calls `get_db()` and provides the yielded session to the endpoint. The dependency cleanup closes the session after the request.

Important database operations

```
db.add(db_job)         # Stage a new ORM object
db.commit()            # Commit transaction to PostgreSQL
db.refresh(db_job)     # Reload saved values from database
```

- `add()` stages an object in the session.
- `commit()` commits the transaction, saving the change to the database.
- `refresh()` reloads the object from the database.

A helper called `serialize_job(job)` converts ORM objects into dictionaries containing `id`, `type`, `payload` and `status`. This keeps the API response format consistent.

---

7. API endpoints implemented

Method + path	Purpose	Success
`POST /api/v1/jobs`	Create and save a job	202 Accepted
`GET /api/v1/jobs`	List jobs using skip and limit	200 OK
`GET /api/v1/jobs/{job_id}`	Retrieve one job by ID	200 OK
`PATCH /api/v1/jobs/{job_id}/status`	Update a job status	200 OK

HTTP status codes

- **202 Accepted:** the job has been accepted; it does not mean processing has finished.
- **200 OK:** the request succeeded. The status update returned HTTP 200.
- **404 Not Found:** returned when the requested job ID does not exist.

---

8. Testing and persistence verification

The endpoints were tested in FastAPI Swagger UI at `http://127.0.0.1:8000/docs`.

Create a job

```
{
  "type": "report_generation",
  "payload": {
    "topic": "Artificial Intelligence",
    "format": "pdf"
  }
}
```

The API returned HTTP 202, a unique job ID and status `queued`.

Update the job status

```
{
  "status": "processing"
}
```

The PATCH request returned HTTP 200 and the response body showed status `processing`.

Persistence test

1. Stop Uvicorn with Control + C.

2. Restart it with `uvicorn app.main:app --reload`.
3. Call `GET /api/v1/jobs` in Swagger.
4. Confirm the same job is still listed with status `processing`.

The job remained present after the server restart. This proves the API is using persistent PostgreSQL storage rather than an in-memory dictionary.

---

9. Day 3 completion checklist

Checkpoint	Result
PostgreSQL connection test	Passed
Job model import	Passed
`jobs` table created and verified	Passed
Database session dependency	Passed
Create, list, retrieve and update endpoints	Tested
HTTP 200 status update	Passed
Persistence after server restart	Passed

What I learned

1. An ORM model represents a database table.
2. A session is the unit of work used to query and change records.
3. A transaction must be committed for changes to be saved.
4. FastAPI dependencies can provide and clean up resources such as database sessions.
5. Restarting the server is a practical test for persistent storage.

Before Day 4: Be able to explain this flow: HTTP request → FastAPI endpoint → `Depends(get_db)` → SQLAlchemy session → PostgreSQL → response serialization. Review `add()`, `commit()`, `refresh()`, `query()`, `filter()`, `first()`, `all()`, `offset()` and `limit()`.

Production note: This is a learning-stage implementation. Before production, move `DATABASE_URL` into environment configuration, never commit secrets, add automated tests and migrations (for example, Alembic), and improve database error handling.